

Evaluation of Cargo Scan and LLM-based Rust Crates Auditing

Yilin Fang and Horng-Ruey Huang

Our project consists of two parts: **evaluation of Cargo Scan** and **LLM-based Rust crates auditing**. We have attached the screen recording, source code, experience spreadsheet, and project report for the final project submission.

Evaluation of Cargo Scan

In our initial plan, we aimed to evaluate Cargo Scan and identify potential issues in its analysis. To this end, we conducted experiments on 19 widely used Cargo crates, the results are shown in the following table.

	Line of code	Number of effects	Execution time
proc-macro2	989	5	6 min
quote	506	1	4 min
base64	554	19	4 min
rand	259	4	2 min
toy-crates	100	7	1 min
resolution-ex	128	2	1 min
recursion-ex	30	4	30 sec
permissions-ex	25	7	30 sec
parsing-ex	137	1	1 min
num_cpus_minimal	43	2	30 sec
libc-ex	16	2	20 sec
fns-closures	19	5	20 sec
ffi-ex	119	16	1 min
dependency-parent	14	1	20 sec
dependency-ex	24	3	30 sec
cfg-ex	60	3	40 sec

caller-checked	55	4	40 sec
syn	4179	177	10 min
hashbrown	4630	277	10 min

In terms of performance, we find that the effect analysis of Cargo Scan is highly efficient. Most crates can be analyzed within one minute. Even for relatively large codebases, such as syn and hashbrown, each containing more than 4,000 lines of code, the analysis completes within approximately 10 minutes.

Regarding accuracy, we initially aimed to identify false positive cases and uncover potential issues in Cargo through the reported positive samples. However, after manual inspection, we did not observe any false positives among the analyzed results.

LLM-based Rust Crates Auditing

Although Cargo Scan provides accurate effect analysis, it still requires human effort to audit the safety of identified unsafe effects. Our goal is to leverage a large language model (LLM) to replace this manual effort and automate the entire process.

The agent takes an effect analysis record produced by Cargo Scan as input. Each record includes the crate name, the enclosing function declaration, the callee, the effect type, and the source location. Instead of loading the entire crate into the context window, the agent relies on a set of predefined tools to retrieve relevant code snippets, either by function name or by code position. Once sufficient evidence is collected, the agent produces a verdict along with a concise rationale.

It is worth noting that, to avoid relying on search or grep-based tools for retrieving code by function name, we use Tree-sitter to parse the entire codebase in advance. The extracted function names and their corresponding code are then stored in a SQLite database. This design significantly reduces both retrieval time and token consumption.

We analyzed eight crates from RustSec Advisory Database. The table below shows the number lines of code and the number of effects for different crates. Cargo Scan significantly reduces the auditing work from more than hundreds lines of code to less than 40 effects. The total number of effects is 92 including 46 safe effects and 46 unsafe effects. The ground truth of safe effects and unsafe effects are made by a manual auditor.

Crates	LoC	Number of Effects
Abomonation	677	39
Abox	220	2
Acc_reader	403	9
Adler	431	2
Alg_ds	880	3
Ammonia	4812	14
Adtensor	114	3
Embedded_alloc	378	20

We select five LLMs (Minimax m2.7, Qwen 3.6 plus, Kimi k2.6, GLM 5.1, GPT-5.4, GPT-5.5) to make the final decision on the safety of effects. The accuracy of auditing effects using LLMs are shown as the below table. In the table, GPT.5.5 achieves 79% accuracy with the most number of correct unsafe cases, but LLMs are not quite as good as the auditor. Because GPT-5.5 supports a very long context window, we choose the entire file as the input of GPT-5.5 to satisfy the requirement of information. Other agents cannot receive sufficient context information due to the budget limitation. Minimax m2.7, Qwen 3.6 plus, Kimi k2.6, and GLM 5.1 have more corrected safe cases than corrected unsafe cases. This illustrates that LLMs do not have enough training data to judge the safety of effects.

	Safe	Unsafe	Acc.
Minimax m2.7	34	4	38/92
Qwen 3.6 plus	40	3	43/92
Kimi k2.6	28	7	35/92
GLM 5.1	36	8	44/92
GPT-5.4	24	25	49/92
GPT-5.5	37	36	73/92

Conclusion

The effect analysis in Cargo Scan demonstrates high efficiency and accuracy, processing large codebases in a short time without producing false positives in side-effect detection.

By comparison, LLM-based auditing offers limited benefits in classifying crate safety. Constraints such as budget and context window size pose significant barriers to achieving performance comparable to expert auditors.